Exploring the Kaleidoscope Hypothesis: Executable Abstraction Discovery in ARC-AGI-2

Len Yabloko*

Abstract

Introduced by Chollet in 2019, the Abstraction and Reasoning Corpus (ARC) has evolved from static generalization, to compositional generalization, to interactive skill acquisition in ARC-AGI-3 [6]. Chollet's informal *kaleidoscope hypothesis*, explicitly named in his 2020 talk, proposes that diverse situations arise from compositions and transformations of a relatively small stock of abstractions [4, 5]. We ask whether this reusable structure can provide a basis for recursive abstraction discovery and, ultimately, interactive skill acquisition. Following DreamCoder's wake–sleep organization [12], we propose alternating inference over retained knowledge (wake) with abstraction learning (dream). DreamCoder jointly learns a library of reusable program abstractions and a neural model that guides synthesis on new tasks. We instead ask large language models (LLMs) to interpret scenes and express candidate abstractions as object-level programs. We build on ARGA and its extension in NSA, which separate graph abstraction from program search [8, 9]: a perception stage converts grid scenes into abstract graphs, where nodes represent candidate objects with their properties, and edges encode relations between them. The language model interprets these structured observations and expresses object recognition and transformation rules in the Guarded Object Language (GOL). Our contributions connect acquisition, composition, retention and evaluation of reusable abstractions through an executable interface (Section 3). GOL definitions make acquired behavior available as callable building blocks for new task solutions. Our experiments test acquisition, retention, cross-task reuse and portability across control interpreters (Section 5); the results establish bounded instances of these capabilities (Section 6).

The experiments test individual mechanisms on public training tasks. Graph-assisted LLM readers generate executable object transformations that reproduce demonstrations withheld during program construction. A retained action also preserves its behavior across controlled variations within a task family, while adding another action preserves earlier demonstrated behavior. A reader-generated program directly invoked a saved action that extends an object to the lower grid boundary to construct part of a cross on a different task. Replacing identical action bodies with calls to that definition exposed the same reusable structure in other generated solutions. The acquired action thus becomes a building block of a new solution. A separate submitted notebook performs LLM-free rule induction and bounded program search over a frozen prior vocabulary, with a Kaggle public leaderboard score of 2.50 (Section 6.5). This score does not evaluate the new GOL acquisition-and-reuse approach. We propose transfer, conversion-yield and cost measures to assess whether retained knowledge improves subsequent solving.

*The author used commercial LLMs to assist in conducting all experiments and in writing this paper. This assistance is distinct from LLM calls within the evaluated procedures; designated wake execution and deterministic checks run without such calls. The manuscript is released under Creative Commons Attribution 4.0 (CC BY 4.0). Repository source code is provided under the MIT License, with existing copyright notices preserved; third-party dependencies retain their respective licenses. Public replay package: `https://github.com/lenyabloko/arc-gol-paper-replay`.

1 Introduction

An ARC solver can produce a correct answer without leaving behind knowledge that helps it solve another task. This distinction matters as tasks demand deeper compositions and, in interactive settings, actions that reveal unfamiliar environmental rules. Solving each case through renewed search may improve a competition score while leaving the cost of acquiring the next skill unchanged. The research problem addressed here is how experience can yield reusable abstractions, and how to distinguish that gain from additional computation or prior exposure.

The living ARC survey identifies discovery of a system's own primitive vocabulary as a possible next step beyond refinement of individual solutions [6, Section 5.3.3]. This raises an acquisition problem: which recurring structures should become reusable components, under what conditions should they apply, and how can new components be retained without invalidating earlier ones? The kaleidoscope hypothesis motivates this recovery problem: even if diverse situations share a compact vocabulary of abstractions, a learner still needs to discover that vocabulary, recognize where each component applies, and learn how to combine its components. Our proposal draws on the extensive priors learned by language models during pretraining on massive, diverse text corpora to suggest candidates, graph observations to ground their meaning, and executable checks to support their retention and reuse.

Chollet explicitly names the informal kaleidoscope hypothesis in his November 2020 talk deck [4]: diverse situations can arise from combinations of a relatively small stock of abstractions. Huh et al.'s later Platonic Representation Hypothesis (PRH), introduced in May 2024, proposes that learned representations converge toward a shared statistical model of reality [1]. The connection we explore is whether common structure captured by learned representations can be recovered as the reusable abstractions and composition rules suggested by the kaleidoscope hypothesis. PRH concerns convergence between representations; the kaleidoscope hypothesis concerns the compositional structure of situations. Neither establishes how to extract an executable vocabulary or use it efficiently. We use ARC to connect these distinct questions: can knowledge elicited from language-model priors be recovered as explicit, executable abstractions whose reuse improves subsequent task solving? Recent leading ARC systems achieve strong results through test-time adaptation, program search and iterative refinement [14]. These results establish task-solving capability, but aggregate accuracy alone does not show whether acquired knowledge supports prediction and planning in a new environment. ARC-AGI-3 makes that distinction practical: an agent must learn from interaction which actions change the situation and how those changes advance a goal [7]. An agent encounters new situations in which the same objects and relations appear in different combinations; it must use what it has learned to anticipate action effects and choose what to do next. Retained recognition and transformation rules are explicit candidates for this reusable knowledge.

Our primary proposal is an architecture for recovering and accumulating executable knowledge from pretrained LLMs. It elicits candidate object relations and transformation rules from knowledge acquired during pretraining, grounds them in observed scenes, and retains checked realizations as world-model components for subsequent reasoning. Section 2 relates this recovery mechanism to representation learning and existing program-library approaches.

An LLM can propose operations for the current task without preserving the resulting solution as reusable knowledge: later tasks still need a way to retrieve what was learned, determine where it applies, and compose it with other constructs. ARGA and NSA supply the graph abstractions and object operations that form our perceptual and execution basis [8, 9]. We add a persistent vocabulary of learned recognition and transformation constructs, a controller that selects and composes those constructs, and a container that gives the resulting programs defined execution semantics (Section 4).

This architecture operates in alternating learning and execution phases, following DreamCoder's wake–sleep organization [12]. During learning (dream), the LLM proposes candidate constructs grounded in scene evidence; checked definitions and their support records may be admitted to the store. During inference (wake), the controller composes constructs from a fixed store version and the container executes the resulting GOL program. The trace supplies evidence for a subsequent learning cycle without changing knowledge during execution. Recursive discovery would require a newly learned composition to become a retained abstraction that supports a subsequent learning cycle. For example, a downward-extension action can become part of a cross-construction routine; that routine would then need its own applicability conditions and supporting evidence before admission as a reusable construct. Guarded Kleene Algebra with Tests (GKAT) would express these applicability conditions as Boolean guards on composition [11]. For the cross routine, an entry guard could check that the bound object has the required marker role and that the planned drawing preserves objects required to remain unchanged. Within its downward-extension component, a loop guard tests whether the object has reached the lower boundary; the container reevaluates that test after each extension. Later components similarly test their conditions on the updated graph (Section 4.2). GKAT supplies the guarded control structure; acquisition must still identify and justify the scene predicates that give those guards meaning.

ARC makes this proposal testable at two levels. Exact output comparisons check a program's predictions on particular cases. Subsequent tasks and independently specified changes to object relations test whether retained knowledge remains useful when appearances or circumstances change. The initial study uses static grid tasks to isolate these effects.

We also examine consistency across transformed views of a task. An *orbit* is the set of views obtained under a specified group of transformations, such as rotations, reflections or colour permutations. A rule should preserve its intended meaning under transformations appropriate to the task, although its coordinates or directional predicates may change. Representation coverage and object correspondence must be checked before attributing disagreement across views to a rule.

The proposed study asks whether graph observations improve acquisition at a fixed budget, whether retained abstractions improve later solving beyond the same model and primitives, and whether these benefits survive retrieval costs and checks of earlier support. Comparing grids alone with graph-assisted observations, each with and without acquired definitions, separates perception from retention. Following the prior-work comparison, we summarize the contributions, develop the approach, specify the experiments, and discuss their results and limits.

2 Prior Work

LeCun's proposal uses Joint-Embedding Predictive Architectures (JEPAs) to learn predictions in representation space as a foundation for world modeling and planning [13]. Balestriero and LeCun's recent PRH study establishes alignment under specified conditions on paired observations and tests the sample efficiency of that alignment [2]. It addresses how shared representations can be learned efficiently; it does not by itself establish their sufficiency for acquiring and transferring action rules. Our approach instead recovers explicit object relations and action rules from pretrained LLMs and makes them executable through a symbolic interface. These acquisition mechanisms can be complementary; their practical comparison requires a matched experiment.

The choice of paired views makes the distinction concrete. Balestriero and LeCun's "good views" are paired observations of one scene under the assumptions of their alignment theorem [2]. Our orbit views instead provide controlled tests of executable meaning: rotating an enclosure preserves containment but changes directional relations, so a rule must transform consistently rather than

produce identical coordinates. An orbit is therefore not automatically a source of good views in the theorem's sense. Likewise, a grid and its derived object graph supply paired descriptions with a human-designed structural bias, not independently obtained observations. The complementary empirical question is whether such descriptions help recover rules that remain consistent across appropriate transformations and reusable across tasks.

The generalized planner GPAR uses the Planning Domain Definition Language (PDDL) and external object-abstraction functions to search generalized ARC plans with pointers, restricting actions, predicates, arguments and program structure [10]. Our proposed difference is acquisition and retention of recognition conditions and parameterized realizations for later tasks, rather than loops or object-level planning themselves. GOL dispatches predicates and effects through graph adapters; perception, primitive semantics and the initial vocabulary remain supplied.

Executable skill libraries and LLM-assisted library learning are established directions. Voyager accumulates callable code skills through interaction and feedback [20]; LILO combines LLM-guided synthesis with Stitch compression and documentation of reusable functions [21]. In ARC, CodeIt improves a program-generating model through prioritized hindsight replay [22]; Hypothesis Search turns natural-language hypotheses into programs checked on demonstrations [23]. BARC combines explicit program inference with direct output prediction, showing that these routes can be complementary [24]. Our proposed distinction is the combination of graph-grounded object roles, a guarded execution interface, explicit applicability and preservation evidence, and separate measurements of acquisition and external reuse. We do not claim novelty for storing generated programs or using an LLM to propose abstractions; the experiments below test only bounded parts of this combination.

3 Contributions

We propose a route from LLM-assisted ARC solving to retained executable knowledge. The contributions concern the unit of knowledge, its use in reasoning, its preservation and portability, and the evidence required to establish a benefit.

3.1 Acquisition of reusable recognition and action knowledge

We propose retaining an abstraction together with the conditions that recognize its roles, a parameterized GOL realization and evidence of its behavior. This connects the interpretation of a scene to an executable plan that can be invoked on other objects. Relative to searching within a supplied planning domain, the intended advance is to acquire reusable recognition and transformation compositions above that domain and carry them into later tasks. The contribution does not lie in introducing guarded programs or object-level planning; it lies in making their acquisition and subsequent applicability explicit and testable (Section 4.1).

3.2 Condition-guided composition and accountable retention

We propose using applicability structure to restrict which retained constructs and bindings enter program search, while preservation contracts identify what must be rechecked after an action or store revision. This could replace repeated primitive-level search with shorter compositions and prevent additions from silently invalidating earlier knowledge. The experiments check whether acquired bodies remain executable within composed programs; learned applicability and lattice-guided retrieval remain proposed mechanisms (Section 4.4).

3.3 An exportable interface for executable knowledge

We propose exporting definitions and dependencies through a declared object-and-action interface, so that another consumer can supply compatible perception and execution adapters. The intended gain is reuse of acquired knowledge without repeating its LLM acquisition or rewriting it for each target. This offers a behavioral interoperability test complementary to representation alignment: do role bindings, effects and useful task-solving behavior survive the change of consumer? Adapter costs and semantic compatibility are part of the claim, not hidden prerequisites (Section 4.5).

3.4 Evaluation of cumulative and external reuse

We connect these mechanisms to forward and backward transfer, learning cost, conversion yield and external reuse. A crossed experiment separates graph-observation benefits from acquired-store benefits; additional no-LLM and independent-consumer tests establish how much capability remains executable after acquisition and export. These are proposed companions to leaderboard accuracy, allowing a system that accumulates useful knowledge to be distinguished from one that repeatedly solves tasks without retaining it. Section 5 defines the comparisons and denominators; Section 6 separates completed execution checks from unmeasured learning outcomes.

3.5 An offline competition submission

Our submitted ARC notebook[2] deploys a frozen prior vocabulary for offline solving without language-model calls. It induces task-local decision rules over object attributes and uses bounded program search, including composition, to generate candidate outputs. The new GOL study examines how callable definitions can be acquired, preserved and made available to later search. Its acquired definitions are not part of the submitted notebook. Section 6.5 reports the submitted notebook's public leaderboard score separately from the GOL pilot findings. Section 4.3 describes the submitted inference path and its relation to the proposed learning loop.

4 Approach

Following ARGA and NSA [8, 9], we ground GOL programs in ARCGraph object graphs constructed from segmentation and background hypotheses (the public perception implementation). ARCGraph represents objects as graph nodes with properties and relations; alternative segmentations remain candidate interpretations of the same scene. Perception and primitive semantics stay fixed during execution; learning changes the retained recognition and action definitions above that interface. Figure 1 shows how dream acquisition and wake composition share the GOL container.

Acquired knowledge needs an executable interface to the reasoning container. GOL is the embodiment of that interface. Its definitions have parameters for object roles and values; its programs combine predicates and object actions through sequencing, guarded branches and loops. The container parses and expands these definitions, binds role parameters to graph-node handles, maintains execution state, and dispatches primitive operations through graph adapters. A retained construct is therefore callable program structure with applicability and support records, not merely a natural-language label or an additional solver module. The store preserves definitions, the controller proposes their use, and the container executes them. The interpreter and graph-operation

[2] `https://www.kaggle.com/code/lenyabloko/situation-lattice-for-arc-agi-2?scriptVersionId=358393778`.

path can be checked separately from acquisition and from the proposed automatic admission and selection mechanisms.

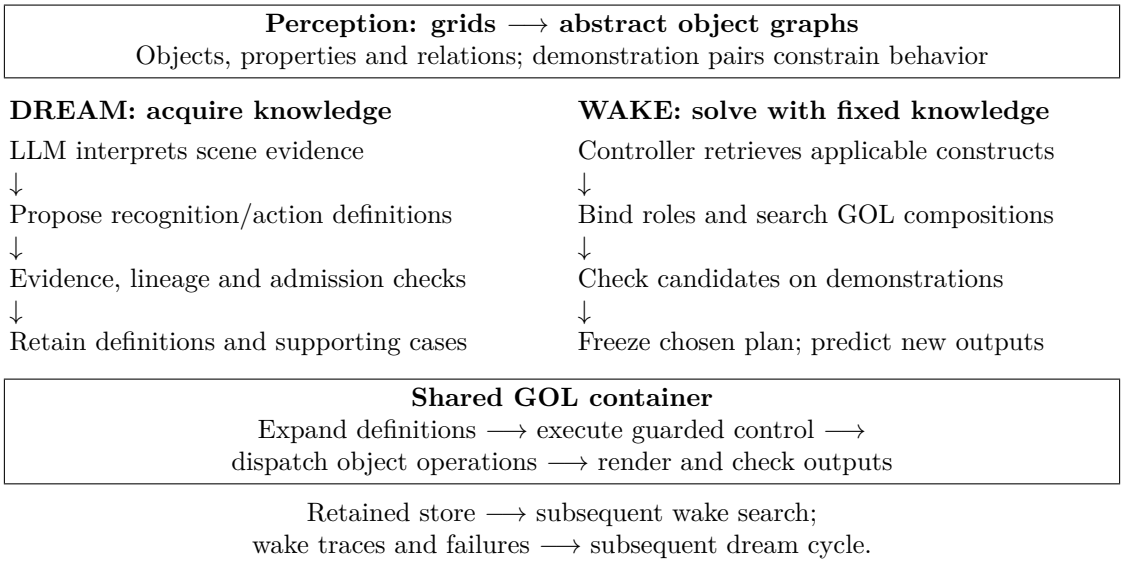


Figure 1: Proposed learning and composition architecture. Dream changes knowledge only through admission; wake reads a fixed store version. The submitted notebook performs rule induction and bounded search over frozen priors. Bounded composition search exercises acquired GOL bodies; beneficial reuse across tasks and cumulative learning remain to be evaluated.

4.1 Planning and reasoning with executable abstractions

An executable abstraction is $K = (D, C, P, E)$: a description D, applicability conditions C over typed roles, a parameterized GOL realization P, and supporting cases E. A binding θ assigns roles to graph objects and values. If $C(s, \theta)$ holds in scene state s, execution of $P\theta$ produces a successor state or an explicit failure. Supporting cases record inputs, bindings, expected results and provenance; they do not establish a universal specification. We use *abstraction* for this semantic package, *definition* for a callable GOL body implementing its realization, and *construct* for a retained abstraction record. Persisting a definition alone does not establish the full abstraction: its applicability conditions still require evidence.

A *contract* accompanies a GOL definition: it states the conditions under which a call is admissible, the effects it may have, and the facts it must preserve. The contract specifies requirements; a *guard* is the executable predicate that controls whether a call or loop step proceeds. In the proposed controller, binding the roles turns the applicability condition C into an entry check on the current scene. A false or unresolved condition blocks that candidate call. After execution, effect and preservation information determines which facts can be retained and which must be recomputed before the next guard. During acquisition, candidate conditions require supporting evidence, while effect and preservation guarantees require justification from the underlying operation semantics. The present examples execute GOL guards; automatic acquisition and enforcement of complete contracts remain proposed mechanisms.

Task programs arise in two ways in the proposed architecture. During learning, an LLM acts as a *reader*: it interprets the demonstrations and proposes object actions together with a program that uses them. Checked actions can be retained as callable definitions. During wake execution,

the controller binds retained definitions to scene objects and searches compositions of those calls and supplied primitives. The container executes the resulting task program in either case.

For example, in the definition-acquisition experiment (Section 5.4), a reader proposed downward extension as a callable action within a program that separates seed components and iterates over output objects. The retained action body is:

```
fill_down(obj) :=
  (while (< (bottom out obj) (- (H out) 1))
    (extend obj 2))
```

Here `out` denotes the output graph, `obj` binds one of its objects, `bottom` gives the object's lowest row, and `H` gives grid height. Rows are indexed from zero; direction 2 denotes downward extension. The container repeatedly dispatches the action through the graph adapter until the object reaches the bottom boundary. The definition supplies the realization P. Its loop guard asks whether the object has reached the lower boundary; it controls continuation after the action has been selected. A contract-derived entry guard would instead check whether downward filling is appropriate for the bound object in this task. The loop condition alone does not establish that applicability.

Relations must also be reconsidered after transformation: moving an object can invalidate its containment in an enclosure even though the object's identity persists. Different graph states can also render the same grid: an object outside the canvas remains in graph state but contributes no visible cells. Exact grid agreement therefore leaves object-level interpretations underdetermined; binding and effect traces retain distinctions hidden by rendering.

4.2 Composition in GOL and its guarded-control foundation

GOL combines retained definitions and primitive actions through sequence $P; Q$, guarded choice if b then P else Q, and iteration while b do P. Subprograms communicate through graph state and explicit bindings. In the evaluated container, definition calls expand by substitution into shared registers—named locations holding object handles and values; parameters do not create isolated local scopes. Composition must therefore account for register reads and writes as well as graph effects. Each guard is evaluated after the preceding actions; syntactic composition alone does not establish that the next component applies. A discovered composition may become a callable construct after admission.

The preservation part of the contract in Section 4.1 addresses the frame problem: which facts remain true after an action? Moving an object may preserve its colour and shape while changing its position and containment relations. Composition may carry forward only facts justified by the operation semantics; affected relations must be recomputed before the next guard. Execution traces and replay check the implementation of these obligations.

For a fixed, finitely expanded realization with abstract actions and pure Boolean tests, Guarded Kleene Algebra with Tests (GKAT) supplies a linear-size deterministic automaton and an equivalence algorithm taking $O(n\alpha(n))$ time for combined term size n, with a fixed Boolean atom set [11, Proposition 4.2 and Theorem 5.10]. An atom assigns truth values to the primitive tests; α is the inverse Ackermann function. This concerns equivalence checking, not the cost of finding a program. Applying it to GOL requires translation into this abstract guarded-control form without changing behavior, together with defined guards and explicit failure semantics.

An equivalence-aware controller could remove redundant control candidates before graph execution, complementing applicability filtering. This is a proposed optimization; neither its implementation nor a search speedup is established by the present results. Practical limits are discussed in Section 7.

4.3 LLM-free wake execution and the submitted notebook

The submitted notebook uses a fixed prior vocabulary without an LLM during solving. It constructs object attributes from candidate graph abstractions, forms lattice concepts from attribute conjunctions, and induces ripple-down-rule (RDR) style decision lists from each task's demonstrations. Candidate rules are applied to test inputs without using test answers. Additional search routes fit whole-grid programs and bounded compositions; description-length criteria help rank candidates. A second pass widens the object-lattice search for unresolved output slots while preserving previously produced predictions. This is task-local inference over frozen priors, not an update to a persistent library between tasks.

The notebook's decision-list and whole-grid program representations are distinct from the callable GOL definitions evaluated here. Figure 1 proposes connecting acquisition to a consumer of those definitions: typed roles restrict bindings, intermediate-state guards restrict actions, and demonstration outputs reject inconsistent candidates. A successful composition can enter the store only through a later admission step.

4.4 Structured retrieval and preservation of knowledge

A *frame* is the retrieval representation of a retained construct, packaging its roles, guards, realization and preservation conditions. For a fixed set of typed predicates, each scene–role binding determines which predicates hold. The formal-concept lattice groups recorded bindings by their shared predicates; a more specific concept covers fewer bindings and shares more predicates.

The preservation contracts defined in Section 4.1 also govern retrieval after an action: the controller retains justified facts, recomputes affected predicates, and matches candidate frames against the updated scene.

Effect contracts can also rule out an unsuitable search vocabulary before composing its members. If every available action preserves the current pixel palette, and control operations introduce no colours, every successful composition preserves that palette. A demonstration requiring a new colour therefore excludes that vocabulary. For a candidate that includes colour-producing actions, the initial palette together with their literal colour arguments gives a conservative set of possible output colours; unrecognized effects remain undecided. This is a necessary-condition check: passing it does not establish that a suitable composition exists. It complements recognition of object roles with a check that the available actions can produce the required kind of change.

4.5 Mechanisms for retention and transferable reuse

Parameterized roles allow a retained relation or action to apply to new objects; callable definitions can replace long primitive sequences with shorter compositions. These mechanisms could improve forward transfer and time to a performance threshold [15, 16], provided retrieval and binding cost less than the search they replace. Comparisons against primitive-only search and a library searched without applicability filtering can test these alternatives.

Dependency tracking and admission replay check revisions against earlier support before replacing the accepted store. This can prevent recorded regressions, but conservative admission may inhibit learning and does not guarantee retention on unseen cases. Independent retention tests, admission cost and rejection rates measure that tradeoff.

A frozen GOL store can support execution without repeating LLM acquisition. Conversion yield measures how much of a reference LLM's target-task capability this consumer recovers; an empty-store baseline tests the store's contribution. Exported roles, predicates, operations and dependencies may also be mapped to an independent consumer [17]. External conversion yield

tests whether reuse survives that mapping, charging adapter construction and reporting semantic mismatches. Section 5 defines how to test whether these benefits accumulate across learning cycles.

5 Experimental Design

The design tests whether the interface proposed in Section 4 connects three capabilities: acquiring an executable object action, preserving it as knowledge, and using it in another program. Each comparison isolates one architectural dependency. The completed studies use selected public training tasks and controlled scene variations; the competition notebook supplies a separate deployment result. Detailed protocols, programs and outcomes are retained in the public replay repository.

5.1 Scene interpretation and executable acquisition

Section 4.1 requires an interpretation of a scene to become an executable object transformation. The acquisition comparison varies the information available to the reader—grids alone or grids with object graphs—while keeping the execution language fixed. The development tasks concern growing objects diagonally and extending coloured seed objects down their columns to the lower boundary. Agreement beyond the demonstrations shown to the reader tests whether the proposed program captures more than those particular examples.

Perception is an architectural dependency, not merely an input format. Executing the same program under different object segmentations tests whether correctness depends on the reader and executor sharing object boundaries. The matched comparisons disclose the same perception contract—the executor's segmentation and background conventions—in every condition. Separate supplied-program controls on diagonal growth and matching loose pieces to cup-shaped regions by capacity establish execution consistency; they do not measure acquisition.

5.2 Composition through the GOL interface

Sections 4.2 and 4.3 require retained definitions to function as callable parts of new programs. A *caller* is a task program that invokes a saved definition. Constructing callers on the tasks used to acquire the actions (the source tasks) tests role binding; five other public training tasks test selection of an acquired action for a different purpose. Each task has four conditions: grids alone or grids with graphs, each paired with an empty store or one containing the acquired actions. Two reader implementations, Codex and Claude Opus 5.5, carry out this comparison; Results calls them the first and second reader, respectively.

Direct calls establish reuse during construction. Exact factoring replaces an identical action body in a generated program with a call to the stored definition, testing shared executable structure even when the reader did not name that definition. Equivalence of expanded programs and execution traces supports that structural claim. LLM-free caller search separately tests whether the controller can compose acquired bodies without another reading.

5.3 Applicability and relational meaning

The retrieval proposal in Section 4.4 requires both suitable object bindings and compatible action effects. Empty-store and acquired-store searches on the same targets test whether adding definitions changes what the controller can solve. Palette-preservation checks distinguish a vocabulary incapable of producing the required change from a search that has not found a suitable composition.

This tests an effect-based prerequisite for retrieval, not complete learned contracts or lattice-guided selection.

Controlled variants of the downward column-filling task test the retained action's relational scope: dimensions, colours and seed configurations change while the intended extension rule is preserved. Stacked colours and alternative segmentations probe where that rule needs additional conditions. Removing colour-based separation of seed objects or replacing boundary-controlled extension with a loop bounded by grid width tests whether the cases discriminate the relevant behavior. A separate cross-task execution diagnostic examines object states as well as rendered outputs, since correct pixels need not imply preservation of the intended object relation.

5.4 Persistence and reuse across consumers

Section 4.5 requires knowledge to survive both store updates and changes of consumer. Separate acquisitions yield the `fill_down` action illustrated in Section 4.1 and a diagonal-growth action that moves an object northeast by a supplied number of steps and then extends it once in that direction. Reproduction of the first action's support after adding the second tests non-interference under a store update. Comparing a standalone program with its stored-action counterpart tests whether retention preserves behavior across scene variations.

An independently implemented control interpreter tests the execution interface by consuming the same definitions without rewriting them. Agreement in outputs and intermediate object actions supports portability of guarded control. Perception and primitive operations remain shared, so this comparison does not test their independent implementation.

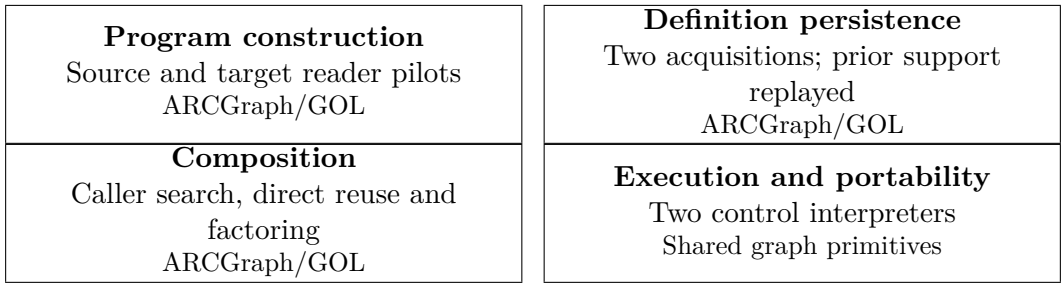


Figure 2: Architectural capabilities tested by the completed experiments. The checks isolate acquisition, persistence, composition and control-interface portability; they are not successive epochs of a completed learning cycle.

5.5 From component checks to cumulative learning

The proposed family-separated study extends the crossed observation/store comparison to successive acquisition cycles. Its central comparison is later-task performance with and without previously acquired knowledge, at matched resources and a fixed execution interface. Acquisition and evaluation families must be distinct, with known exposure recorded. This tests the benefit of an accumulating vocabulary rather than the existence of reusable examples. A separate no-LLM consumer comparison tests how much acquired capability remains executable after the reader is removed.

5.6 Measurements

The following measures connect architectural mechanisms to outcomes. Completed checks support individual instances; longitudinal and conversion-yield measures require the broader study. All comparisons retain their declared populations and use predictions fixed before test-answer scoring.

5.6.1 Observation benefit

The change in successful program construction when graph observations are added at fixed resources measures the observation interface. Equal disclosure of executor semantics is essential; distinguishing graph semantics from extra tokens would also require a matched description control.

5.6.2 Retention benefit

The paired accuracy difference on subsequent tasks between acquired-store and empty-store conditions measures forward-transfer benefit. Actual calls distinguish use of retained knowledge from independent reconstruction. This adapts the evaluation used in Gradient Episodic Memory (GEM) [15]; it measures the effect of an acquired store rather than GEM's transfer before training on a task.

5.6.3 Relational specificity

Preserved behavior under relation-preserving changes, together with sensitivity to relation-changing controls, tests the scope of an abstraction. Expected outcomes must be specified independently of the tested program.

5.6.4 Cost and reliability

Accuracy is paired with acquisition, retrieval, search and adapter costs, including unsuccessful executions. Cost to a fixed accuracy threshold [16] and performance against cumulative acquisition cost distinguish useful reuse from overhead. Equal epoch counts alone do not imply equal effort.

5.6.5 Conversion yield and attributable reuse

For target population T, let L contain tasks solved by the reference LLM and S_K, S_0 those solved by the same no-LLM consumer with acquired and empty stores. Define

$$Y = \frac{|L \cap S_K|}{|L|}, \qquad \Delta_T = \frac{|S_K| - |S_0|}{|T|}.$$

Yield measures recovered reference capability; Δ_T measures the store's contribution. Yield is undefined for empty L. Full-population accuracy and paired wins and losses accompany these measures; reference solutions must not enter the consumer's store.

5.6.6 Retention across learning epochs

Let $a_{e,j}$ denote accuracy at epoch e on fixed evaluation cases for task group j, first learned at epoch j. Mean $a_{e,j} - a_{j,j}$ over $j < e$ measures backward transfer [15]. Earlier correct cases that become incorrect reveal forgetting hidden by aggregate gains. Admission-support replay remains distinct from independent retention evaluation.

5.6.7 External reuse

The conversion and paired-store measures can be repeated with an independent consumer, charging adapter costs and checking agreement in role bindings and effects. Source-support agreement establishes preservation under export; improved target-task performance establishes a stronger reuse benefit.

5.7 Deployment-efficiency scenarios

The cost model asks when reusable execution could reduce reliance on a fallback model. Let a be the fraction of requests handled without fallback, c the cost per fallback request, and r mean routing, execution and checking cost per request. Let A be acquisition and adapter cost, amortized over N requests. Then

$$\bar{c} = A/N + r + (1 - a)c.$$

The projections in Results assume preserved total accuracy, negligible amortized acquisition and $r/c = 0.01$. Acceptance is not correctness: accepted and fallback requests require separate accuracy measurements, and coverage is not conversion yield.

Score density measures exact-accuracy percentage points per megabit of the complete deployed artifact. The illustrative comparison assumes a 1 MB package scoring 1% and a 4B-parameter model stored at 16 bits scoring 24% on the same evaluation. Runtime, dependencies and weights belong within a common accounting boundary. These scenarios motivate cost and compactness measurements; they are not measured advantages of the present system.

6 Results

The findings concern whether an acquired object action can become a building block for subsequent reasoning. Table 1 summarizes acquisition, persistence, composition and portability evidence against the objectives in Section 5. These checks establish concrete instances of reuse; they do not constitute a completed cumulative learning cycle.

6.1 Acquisition and the perception interface

Graph-assisted readers produced executable transformations for diagonal growth and column filling that reproduced the shown and withheld demonstrations (Section 5.1). Reversing demonstration order preserved the diagonal-growth program. This supports the feasibility of expressing scene interpretations through GOL.

The segmentation comparison identifies an important dependency. With the original object segmentation, the graph-assisted plans reproduced every demonstration while the grid-only plans did not; changing to four-connected monochrome objects reversed most of that pattern. The original grid-only prompts omitted the executor's segmentation contract. The result therefore motivates a shared perception contract rather than establishing a general advantage from supplying graphs. The four-condition comparisons in Section 5.2 control that disclosure.

6.2 Persistence and control-interface portability

The acquisition study retained two actions: downward extension to the boundary and parameterized diagonal growth (Section 5.4). Programs calling each action reproduced the corresponding original task test output; adding diagonal growth preserved the downward-extension action's demonstration

Finding	Supporting evidence	Scope of the finding
Acquired actions persist after a store update	Two reader-acquired definitions reproduce both original task test outputs; adding the second preserves all three demonstrations supporting the first.	One addition on two exposed development tasks; no measured improvement on subsequent tasks.
A retained action generalizes within its specified family	The column-filling program and its stored-action version each reproduce all 32 synthetic scenes; a width-bounded composition reproduces 27 and omitting colour splitting reproduces 23.	The same 32 scenes in each comparison; variation within one declared transformation family.
Readers use the retained vocabulary	Three of four store-enabled proposals reference saved actions; all four reproduce the source demonstrations.	Two known tasks; source-task actions are supplied, so this is interface use rather than unseen-task transfer.
A reader composes a saved action into a different task	The second reader invokes the acquired downward-extension action while constructing a cross; the unchanged program reproduces three demonstrations and its test output.	One of ten store-enabled opportunities; direct reuse, without an established accuracy or search-cost advantage.
Definitions preserve behavior across control interpreters	Unchanged definitions and callers preserve all eight outputs after export: six demonstrations and two original test inputs.	A separate control interpreter shares perception and graph primitives; no independent validation of primitive semantics.
A saved action enters LLM-free caller search	Bounded search binds the diagonal action to object size; the caller reproduces all three source demonstrations.	Supplied traversal grammar and an exposed source task; no measured store advantage or unseen-task transfer.

Table 1: Evidence for bounded persistence, within-family generalization, control-interface portability and composition. These findings do not establish improved unseen-task solving. Complete reader outcomes are available in the public replay repository.

support. This shows retention without regression after one addition, rather than an established benefit from repeated learning cycles.

The unchanged definitions and callers also preserved all eight compared outputs under a separate control interpreter. Their behavior therefore survived a change of parsing, expansion and guarded-control implementation. Shared perception and primitive operations limit this evidence to the control interface, as specified in Section 5.4.

6.3 Composition and cross-task reuse

With the perception contract disclosed equally, readers constructed callers of the saved actions on the source tasks. LLM-free caller search independently bound the acquired diagonal action to each input object and used its size as the movement argument, reproducing the source demonstrations. The retained body could thus enter both reader-generated and deterministic compositions (Section 5.2).

The five-target comparison provides evidence beyond source-task reconstruction. Each reader produced fifteen complete demonstration-and-test fits across the five tasks and four conditions, but only the second reader directly selected a saved action: downward extension became part of a cross-drawing routine on a different task. The unchanged routine reproduced the demonstrations and original test output. The same task was also solved without the store, so this establishes direct reuse without an accuracy advantage. Retrospective factoring found the same acquired body inside three first-reader solutions to that task; equivalent expansion and execution establish shared structure, rather than retrieval during their construction. The public records retain all proposals and both forms of reuse evidence.

6.4 Relational scope and applicability

The original column-filling program and its stored-action counterpart each reproduced all 32 controlled family cases (Section 5.3). Omitting the separation of seed objects by colour reproduced 23; replacing the loop that stops at the lower boundary with one that repeats according to grid width reproduced 27. The comparison distinguishes preservation of the intended transformation from source-example fitting. Additional seed configurations and colour renamings preserved behavior, while stacked colours exposed dependence on object segmentation.

6.5 Competition result and its interpretation

The submitted notebook received a Kaggle public leaderboard score of **2.50**. It performs task-local inference and bounded search over frozen priors; the new GOL acquisitions are not part of that submission (Section 4.3). Appendix A records the submission association.

The aggregate score does not measure whether solving leaves reusable knowledge. Retention benefit (Section 5.6.2), backward transfer (Section 5.6.6), conversion yield (Section 5.6.5) and external reuse (Section 5.6.7) address that question. The completed studies demonstrate acquisition and reuse mechanisms, without establishing a benchmark-scale improvement under those measures.

6.6 Resource efficiency and reuse

Table 2 separates published performance from the conditional deployment scenarios in Section 5.7. The published scores use different populations and cannot rank systems under the proposed reuse measures. The projections illustrate potential recurring-cost and storage advantages, conditional

on the stated accuracy and accounting assumptions; no such advantage has been measured for the present architecture.[3]

System or scenario	Reported or assumed accuracy	Reuse, cost or density
Opus 5.5 High (reported)	93.3%, ARC-AGI-2 semi-private	Conversion yield and external reuse unmeasured; shipped size undisclosed
2025 winning test-time training / recursive ensemble (reported)	About 24%, ARC-AGI-2 private	About $0.20/task; conversion yield and external reuse unmeasured
Submitted frozen-prior notebook	2.50, Kaggle public leaderboard	New GOL acquisition not integrated; cumulative reuse unmeasured
Present ARCGraph/GOL study	Full evaluation not performed	Acquisition, retention and a cross-task reuse instance; no demonstrated accuracy benefit
Accepted coverage 25%, 50%, 75% (assumed)	Correct acceptance and total accuracy preserved	Projected cost ratios 0.76, 0.51, 0.26 at 1% routing overhead
Complete 1 MB package (assumed)	1% on a common evaluation	0.125 percentage points/Mbit
4B parameters at 16 bits (assumed)	24% on that same evaluation	At most 0.000375 points/Mbit, excluding ancillary files

Table 2: Published context and conditional projections, kept separate. Sources: ARC Prize [18, 19, 14] and the authenticated Kaggle record for the submitted notebook (Appendix A). Published scores have different populations and cannot establish a ranking under our proposed metrics. Assumptions and accounting boundaries for the projected rows are given in Section 5.7. MB and Mbit use decimal units.

7 Limitations

Graph observations introduce a specific interpretive limitation. A deterministic graph derived from a grid supplies human-designed structural supervision, not an independent observation of the scene.

ARC leaves the intended transformation and its applicability conditions partly unknown. Demonstrations and counterexamples can reject candidates but cannot establish a unique interpretation. The proposal therefore addresses acquisition and retention under incomplete specifications, rather than claiming that another planning language removes this difficulty.

If graphs help current-task fitting but retained definitions do not help later tasks, the result supports an observation interface, not cumulative abstraction learning.

GKAT's nearly linear bound concerns equivalence of already constructed guarded terms with a fixed Boolean atom set, not program synthesis. When that set is allowed to vary, GKAT equivalence is co-NP-hard and lies in PSPACE [11, Proposition 5.11]. Expanding definitions can enlarge the term substantially; evaluating concrete graph predicates, enumerating object bindings and executing primitive actions incur additional costs. A finite control automaton also does not bound integer-register or graph-state storage, establish termination of a loop, or guarantee that tests are defined.

[3]Competition accuracy alone cannot supply conversion yield, retention or external reuse; these require separate measurements.

The practical composer must choose definitions, bindings, order, branches and stopping conditions. Shared scratch registers can interfere between otherwise valid definitions. Definition identity must also distinguish formal parameters from free registers; a name-normalization collision can otherwise confuse different realizations. A composer needs declared register effects or verified separation of private names; renaming registers that intentionally communicate between components would change behavior. These choices can still grow combinatorially with the vocabulary and plan depth, and intermediate transformations can invalidate earlier bindings or relations. Applicability filtering and pruning equivalent guarded-control programs may help, but require sound predicates and a validated translation from GOL; neither establishes useful transfer by itself.

8 Conclusion

We propose a learning and reasoning architecture that recovers executable abstractions from pretrained LLM priors, grounds them in scene graphs, and retains them for reuse through a GOL container. The completed experiments show that generated object actions can persist after a store addition and preserve their behavior under another control interpreter sharing the graph adapter. The second reader also selects a retained action while constructing a solution to a different task; exact factoring identifies the same shared action in other generated solutions.

9 Future Work

The central goal is to establish whether successive learning cycles produce a growing vocabulary of abstractions that improves solving on unseen tasks while preserving earlier capabilities. This requires distinguishing useful compositional knowledge from accumulated task-specific solutions, and determining when reuse reduces total acquisition and search cost.

A second goal is to make acquired knowledge portable: definitions should retain their meaning and practical value when used by an independent reasoning system. Beyond static ARC, we seek to connect this executable knowledge to interactive exploration, prediction of action effects and skill acquisition. This requires choosing actions that reveal unfamiliar environmental rules and using the resulting observations to acquire and revise executable knowledge. A separate interactive evaluation must test whether this process improves skill acquisition. Learned perception, including JEPA-based representations, offers a complementary direction for grounding such knowledge in new environments.

References

[1] M. Huh, B. Cheung, T. Wang, and P. Isola. *Position: The Platonic Representation Hypothesis*. ICML, PMLR 235:20617–20642, 2024. Preprint first submitted 13 May 2024. `https://arxiv.org/abs/2405.07987`; `https://proceedings.mlr.press/v235/huh24a.html`.

[2] R. Balestriero and Y. LeCun. *Proving and Improving the Platonic Representation Hypothesis with JEPAs*. 2026. `https://arxiv.org/abs/2610.39302`.

[3] F. Chollet. *On the Measure of Intelligence*. 2019. `https://arxiv.org/abs/1911.01547`.

[4] F. Chollet. *Abstraction & Reasoning: What Deep Learning Can Do, What It Can't, and What We Can Try Next*. AAAI Fall Symposium talk deck, November 2020; slide 15, "The Kalei-

doscope hypothesis." Date and deck linked from https://fchollet.com/. https://docs.google.com/presentation/d/1yqE5AYS4l9mzn17WjnSKEmvqydgw-CsAYrs_TPClOFA/edit.

[5] F. Chollet. *General Intelligence: Define It, Measure It, Build It.* AGI-24 keynote, 2024. Recording published as *It's Not About Scale, It's About Abstraction*; kaleidoscope hypothesis at 26:10. https://www.youtube.com/watch?v=s7_NlkBwdj8.

[6] S. Vahdati, A. Aioanei, H. Suresh, and J. Lehmann. *The ARC of Progress towards AGI: A Living Survey of Abstraction and Reasoning.* 2026, version 1. https://arxiv.org/abs/2603.13372v1.

[7] ARC Prize Foundation. *ARC-AGI-3.* Benchmark overview. https://arcprize.org/arc-agi/3.

[8] Y. Xu, E. B. Khalil, and S. Sanner. *Graphs, Constraints, and Search for the Abstraction and Reasoning Corpus.* AAAI 37(4):4115–4122, 2023. https://doi.org/10.1609/aaai.v37i4.25527.

[9] P. Batorski, J. Brinkmann, and P. Swoboda. *NSA: Neuro-symbolic ARC Challenge.* 2025. https://arxiv.org/abs/2501.04424.

[10] C. Lei, N. Lipovetzky, and K. A. Ehinger. *Generalized Planning for the Abstraction and Reasoning Corpus.* AAAI, 2024. https://arxiv.org/abs/2401.07426.

[11] S. Smolka, N. Foster, J. Hsu, T. Kappé, D. Kozen, and A. Silva. *Guarded Kleene Algebra with Tests: Verification of Uninterpreted Programs in Nearly Linear Time.* Proceedings of the ACM on Programming Languages 4 (POPL), Article 61, 2020. https://doi.org/10.1145/3371129.

[12] K. Ellis et al. *DreamCoder: Bootstrapping Inductive Program Synthesis with Wake-Sleep Library Learning.* PLDI, 2021. https://doi.org/10.1145/3453483.3454080.

[13] Y. LeCun. *A Path Towards Autonomous Machine Intelligence.* Position paper, 2022. https://openreview.net/pdf/315d43ba26f55357a84cec9a7ed15a6610094f79.pdf.

[14] M. Knoop. *ARC Prize 2025 Results & Analysis.* ARC Prize Foundation, 5 December 2025. https://arcprize.org/blog/arc-prize-2025-results-analysis.

[15] D. Lopez-Paz and M. Ranzato. *Gradient Episodic Memory for Continual Learning.* NeurIPS, 2017. https://arxiv.org/abs/1706.08840.

[16] M. E. Taylor and P. Stone. *Transfer Learning for Reinforcement Learning Domains: A Survey.* JMLR 10:1633–1685, 2009. https://jmlr.org/papers/v10/taylor09a.html.

[17] M. E. Taylor, P. Stone, and Y. Liu. *Transfer Learning via Inter-Task Mappings for Temporal Difference Learning.* JMLR 8:2125–2167, 2007. https://jmlr.org/papers/v8/taylor07a.html.

[18] ARC Prize Foundation. *Claude Opus 5.5: ARC-AGI Results.* Model entry dated 22 September 2026; accessed 8 November 2026. https://arcprize.org/results/anthropic-claude-opus-5-5.

[19] ARC Prize Foundation. *ARC Prize 2025 Competition Details and Winners.* https://arcprize.org/competitions/2025.

[20] G. Wang et al. *Voyager: An Open-Ended Embodied Agent with Large Language Models*. 2023. https://arxiv.org/abs/2305.16291.

[21] G. Grand et al. *LILO: Learning Interpretable Libraries by Compressing and Documenting Code*. ICLR, 2024. https://arxiv.org/abs/2310.19791.

[22] N. Butt et al. *CodeIt: Self-Improving Language Models with Prioritized Hindsight Replay*. ICML, 2024. https://arxiv.org/abs/2402.04858.

[23] R. Wang et al. *Hypothesis Search: Inductive Reasoning with Language Models*. ICLR, 2024. https://arxiv.org/abs/2309.05660.

[24] W.-D. Li et al. *Combining Induction and Transduction for Abstract Reasoning*. ICLR, 2025. https://arxiv.org/abs/2411.02272.

A Submission provenance

The authenticated Kaggle record associates completed submission 57195179 with notebook version 358393778. The source contains task-local decision-list induction, whole-grid search and composition; it does not incorporate the two definitions acquired in this study.